



BERKELEY LAB

Bringing Science Solutions to the World



U.S. DEPARTMENT OF
ENERGY

Office of Science

Parallel Programming with Partitioned Global Address Space Models

Damian Rouson

Computer Languages and Systems Software Group

Parallel Applications Workshop — Alternatives to MPI+X (PAW-ATM), 17 November 2024



Overview

01

Background:
PGAS Models

02

Chapel

03

UPC++

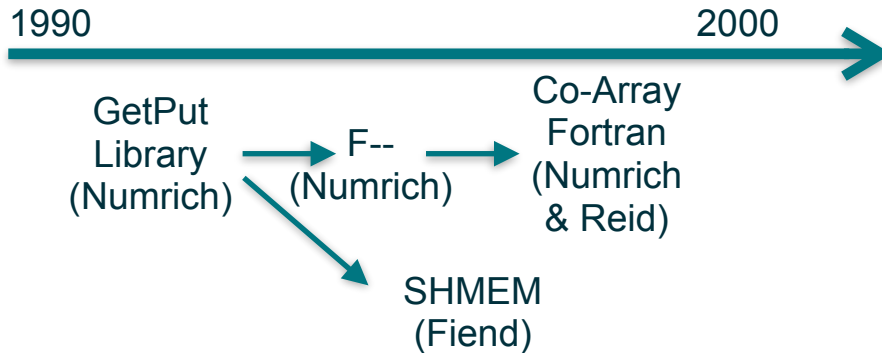
04

Fortran

05

Conclusions

Background



“While not required by the FORTRAN 77 standard, almost all compilers accessed data in a COMMON block based on offsets from the starting address of the COMMON block. In a lucky coincidence, FORTRAN 77 compilers placed COMMON blocks starting at known static addresses. As a result, the instance of the program on each processor had the same address for any particular data item stored in a COMMON block. This feature made it trivial to determine the addresses of a data item in a COMMON block on a remote processor, as it was the same as the address of the corresponding item on the local processor. Memory layout such as this is referred to as symmetric memory. Reid, Long & Steidel (2020)

Top Ten Reasons to Use SHMEM

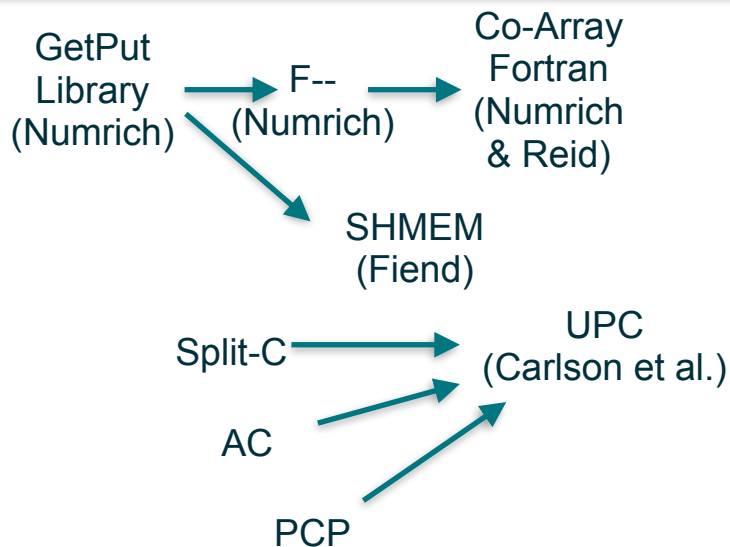
1. Efficient remote data transfer.
2. Ease of use...

Feind (1995)

Background

1990

2000

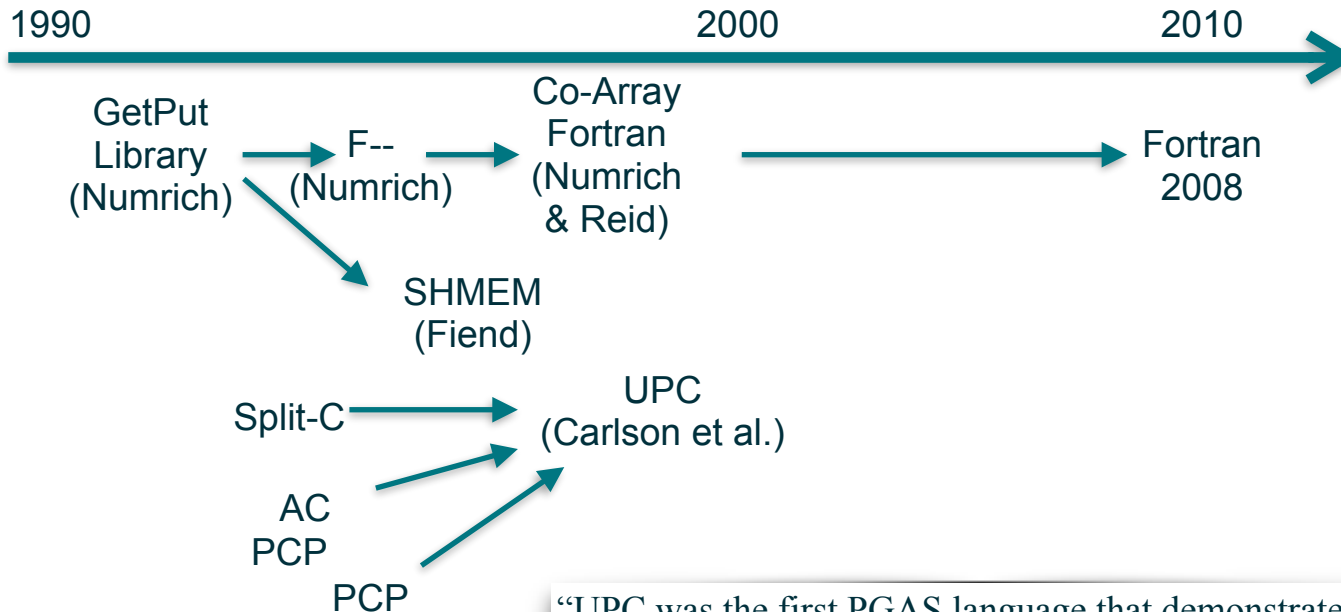


“Even though latency and contention problems in large-scale multiprocessors have resulted in a general move away from uniform shared memory toward distributed memory, the shared-memory programming model retains many attractions for users of these systems. In particular, the ability to read and write remote memory with simple assignment statements is considerably more attractive than having to learn all the conventions of a message-passing library,...

At the same time, the quest for performance often makes it desirable to view program data as distributed among a number of local memories.

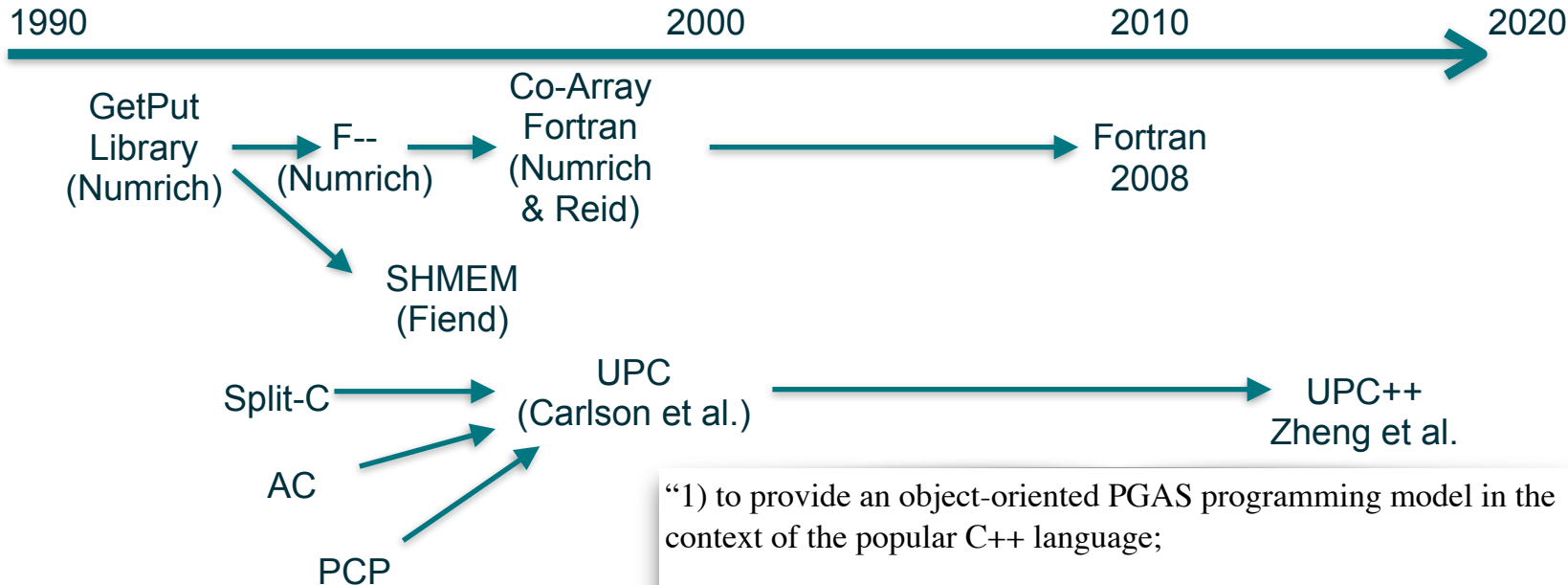
... [UPC] assumes that the programmer must think about issues of memory locality in designing effective data structures and algorithms,...” Carlson et al. (1999)

Background



“UPC was the first PGAS language that demonstrated excellent scalable performance on hundreds of thousands of cores in the HPC Challenge Class 2 (productivity) competition at the Supercomputing (SC) conference in 2005.” Yelick and Zheng (2015)

Background

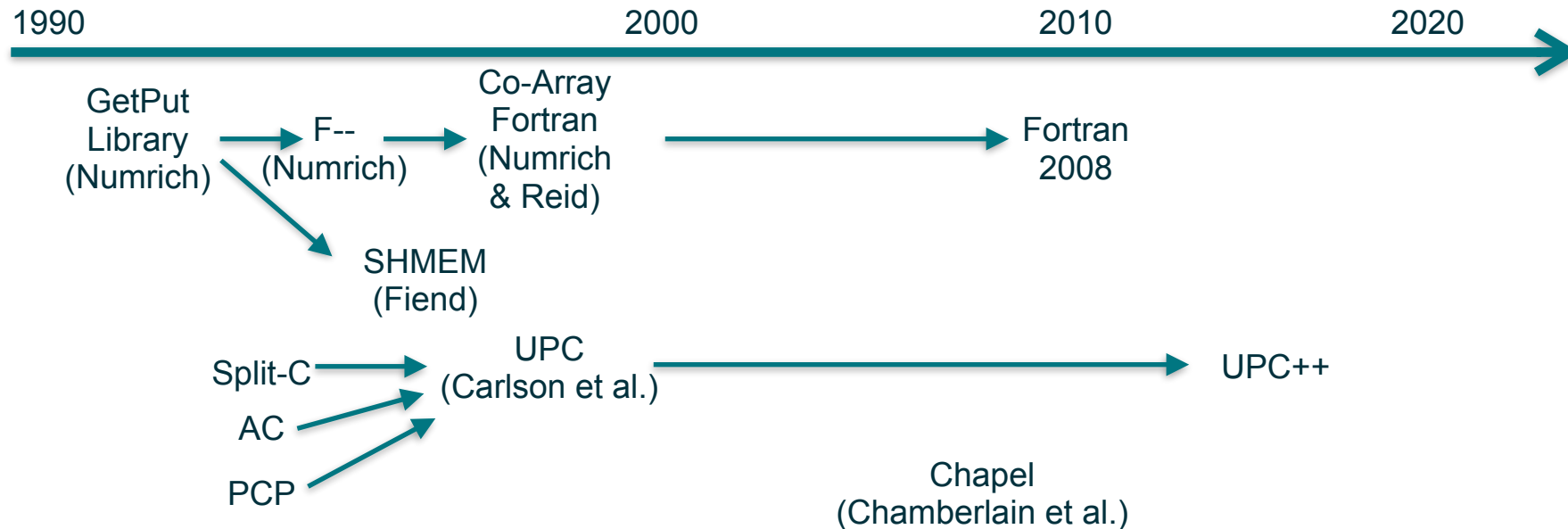


“1) to provide an object-oriented PGAS programming model in the context of the popular C++ language;

2) to add useful parallel programming idioms unavailable in UPC, such as asynchronous remote function invocation and multidimensional arrays, to support complex scientific applications;

3) to offer an easy on-ramp to PGAS programming through interoperability with other existing parallel programming systems (e.g., MPI, OpenMP, CUDA).” Zheng et al. (2014)

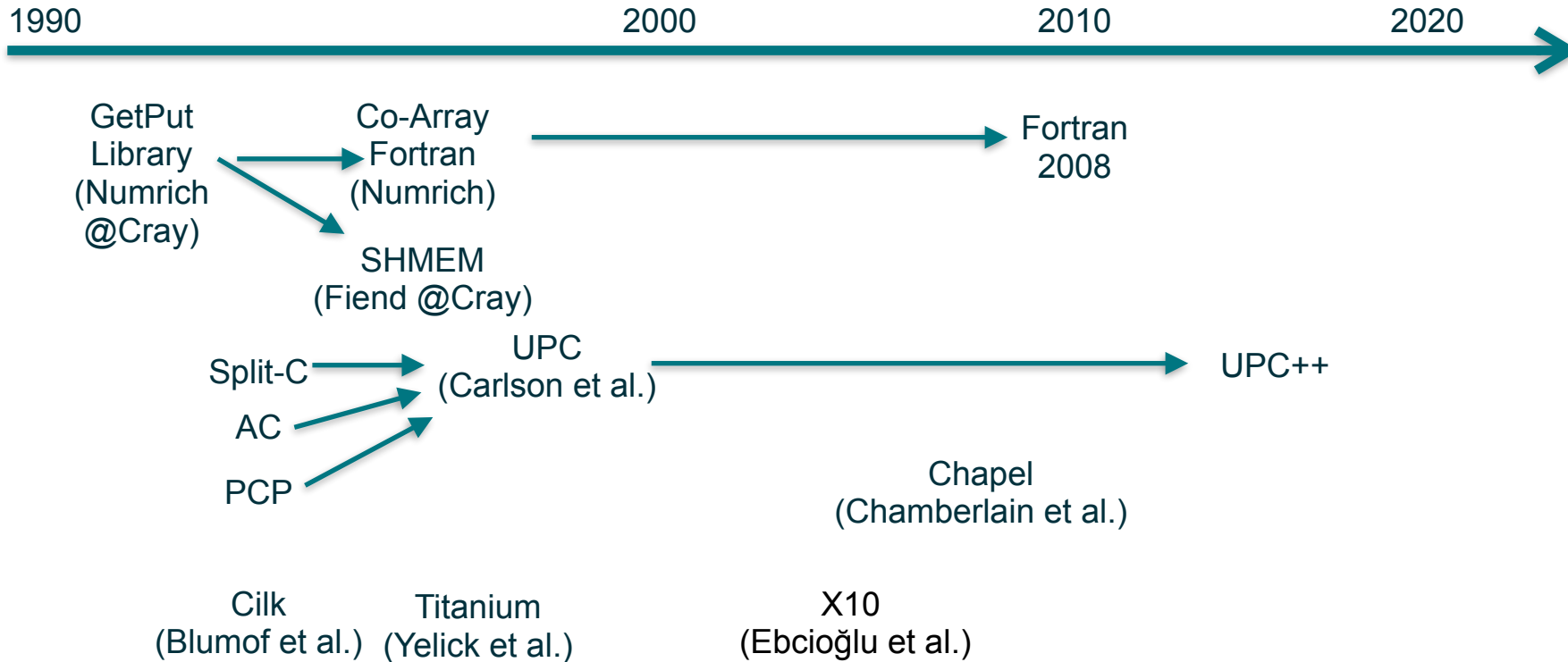
Background



“The parallel computing community’s inability to tap the skills of mainstream programmers prevents parallel programming from becoming more than an arcane skill, best avoided if possible.

... parallel language developers should attempt to develop features that ease the burdens of parallel programming” (Chamberlain et al. 2007)

Background



Chapel Parallel Programming Language

From laptops to supercomputers, Chapel makes parallel programming more productive.

EASY TO USE

As programmable as Python, fast as Fortran, scalable as MPI, SHMEM or UPC, flexible as C/C++

>>>

Get started and leverage your hardware quickly.

PORTABLE

Executes on various platforms: HPE Cray EX, HPE Apollo, CRAY XC, most *nix systems, Mac, NVIDIA and AMD GPUs

>>>

Write your code once and run it anywhere.

FAST & SCALABLE

Achieves 9 TB/s when sorting 281 TB in 31 seconds on 8192 HPE Cray EX Nodes

>>>

Scale your applications with ease.

GPU-READY

Matches performance of CUDA and HIP reference versions of Stream Triad benchmark

>>>

Unlock the power of GPUs for parallel computing.

OPEN SOURCE

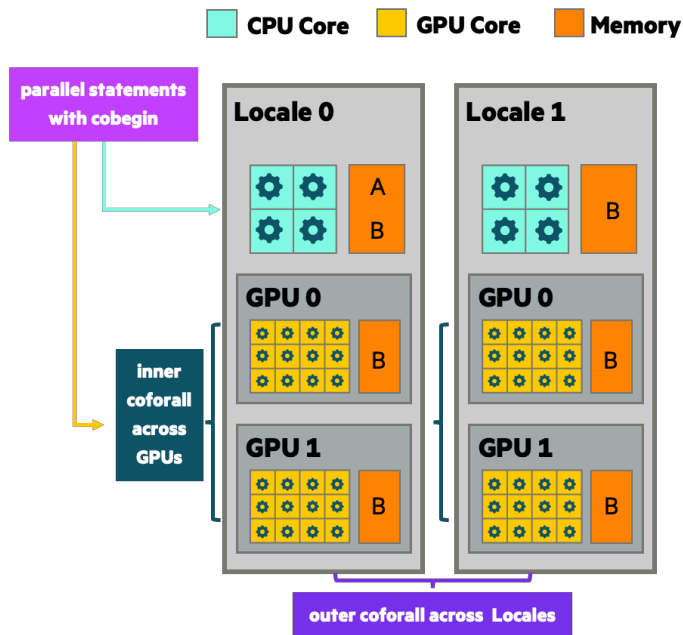
Developed in collaboration with HPE and the Chapel Community

>>>

Join a thriving community of Chapel users and developers!

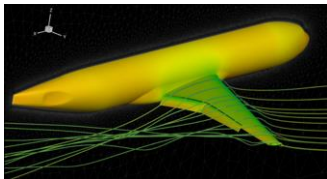
Chapel and GPUs

- Chapel now supports vendor-neutral GPU programming (NVIDIA and AMD)
 - Uses the identical features as for programming multi-core CPUs and HPCs



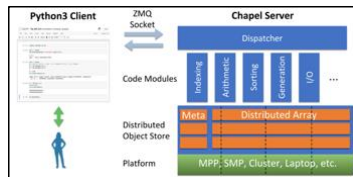
```
var A: [1..n, 1..n] real;  
coforall l in Locales do on l {  
  cobegin {  
    coforall g in here.gpus do on g {  
      var B: [1..n, 1..n] real;  
      B = 2;  
      A = B;  
    }  
  }  
}   
writeln(A);
```

Applications of Chapel



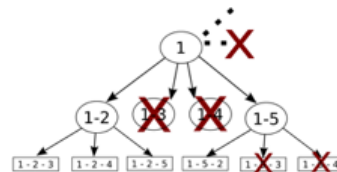
CHAMPS: 3D Unstructured CFD

Laurendeau, Bourgault-Côté, Parenteau, Plante, et al.
École Polytechnique Montréal



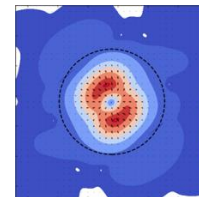
Arkouda: Interactive Data Science at Massive Scale

Mike Merrill, Bill Reus, et al.
U.S. DoD



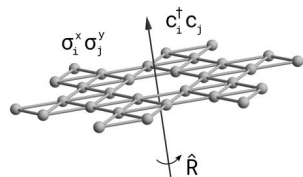
ChOp: Chapel-based Optimization

T. Carneiro, G. Helbecque, N. Melab, et al.
INRIA, IMEC, et al.



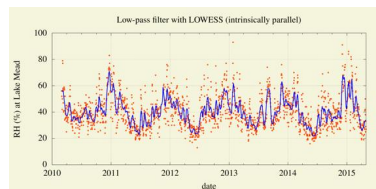
ChpUltra: Simulating Ultralight Dark Matter

Nikhil Padmanabhan, J. Luna Zagorac, et al.
Yale University et al.



Lattice-Symmetries: a Quantum Many-Body Toolbox Desk dot chpl: Utilities for Environmental Eng.

Tom Westerhout
Radboud University



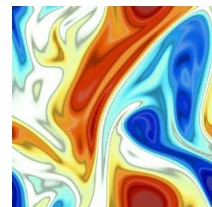
Nelson Luis Dias

The Federal University of Paraná, Brazil



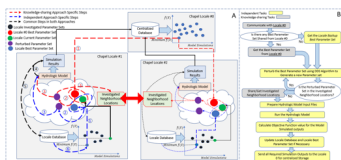
RapidQ: Mapping Coral Biodiversity

Rebecca Green, Helen Fox, Scott Bachman, et al.
The Coral Reef Alliance



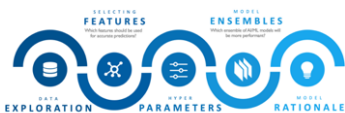
ChapQG: Layered Quasigeostrophic CFD

Ian Grooms and Scott Bachman
University of Colorado, Boulder et al.



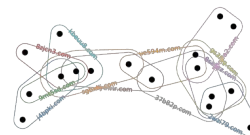
Chapel-based Hydrological Model Calibration

Marjan Asgari et al.
University of Guelph



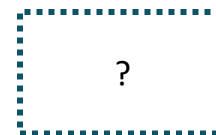
CrayAI HyperParameter Optimization (HPO)

Ben Albrecht et al.
Cray Inc. / HPE



CHGL: Chapel Hypergraph Library

Louis Jenkins, Cliff Joslyn, Jesun Firoz, et al.
PNNL



Your Application Here?

What does UPC++ offer?

Asynchronous behavior

- **RMA:**
 - Get/put to a remote location in another address space
 - Low overhead, zero-copy, one-sided communication.
- **RPC: Remote Procedure Call:**
 - Moves computation to the data

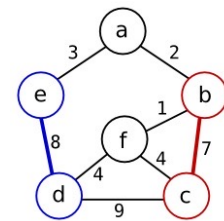
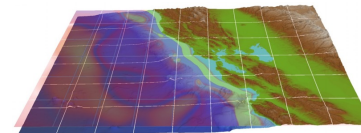
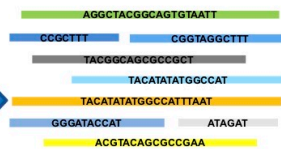
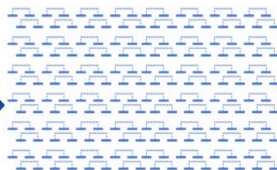
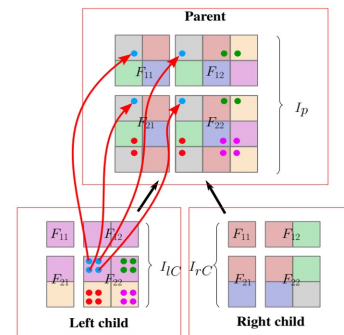
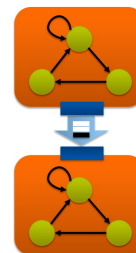
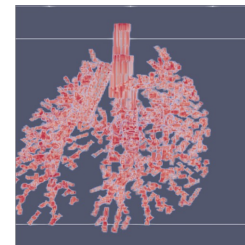
Design principles for performance

- All communication is syntactically explicit
- All communication is asynchronous: futures and promises
- Scalable data structures that avoid unnecessary replication

UPC++ applications

UPC++ has been used successfully in several applications to improve programmer productivity and runtime performance, including:

- symPACK, a sparse symmetric matrix solver: PAW-ATM tomorrow
- SIMCoV, agent-based simulation of lungs with COVID
- MetaHipMer, a genome assembler
- Actor-UPCXX, used in the Pond tsunami simulator
- A UPC++ backend for NWChemEx/TAMM
- UPC++ DepSpawn, a library for data-flow computing
- Mel-UPX, half-approximate graph matching solver



Coarrays: Fortran's PGAS Data Structure



BERKELEY LAB

Bringing Science Solutions to the World

- Distributed data structures — greeting
- Facilitate Remote Memory Access (RMA) — line 15

```
1 program main
2   !! One-sided communication of distributed greetings
3   implicit none
4   integer, parameter :: max_greeting_length=64, writer = 1
5   integer image
6   character(len=max_greeting_length) :: greeting[*] ! scalar coarray
7
8   associate(me => this_image(), ni=>num_images())
9
10    write(greeting,*) "Hello from image",me,"of",ni ! local (no "[ ]")
11    sync all ! image control
12
13    if (me == writer) then
14      do image = 1, ni
15        print *,greeting[image] ! one-sided communication: "get"
16      end do
17    end if
18
19  end associate
20 end program
```

Fortran's Statement/Loop-Level Parallelism



BERKELEY LAB

Bringing Science Solutions to the World

In addition to the SPMD/PGAS features that work in shared or distributed memory, several features facilitate expressing unordered sets of calculations amenable to multithreading, vectorization, or accelerator offloading:



`do concurrent + pure procedures, including elemental procedures`

```
9      integer row, col
10     integer, parameter :: window=4, time=1
11
12     associate(rows => size(distance%body,1), cols => size(distance%body,2))
13     do concurrent(row=1:rows, col=1:cols)
14         associate(first_row => max(1, row-window), last_row=>min(row+window, rows))
15             distance%body(row,col) = minval(hypot( &
16                 this%body(first_row:last_row, time) - rhs%body(row, time), &
17                 this%body(first_row:last_row, col) - rhs%body(row, col) &
18             ))
19         end associate
20     end do
21 end associate
```



`where statement`

```
44
45     where(rhs_filtered/=0._rkind)
46         distance%body = distance%body/rhs_filtered
47     elsewhere
48         distance%body = 0.
49     end where
50
```



Array statements + elemental procedures (intrinsic or user-defined):
`matmul, reduce, transpose, dot_product, merge, pack, unpack, count, any, all, findloc, ...`

CAF at Scale: Magnetic Fusion



BERKELEY LAB

Bringing Science Solutions to the World

Multithreaded Global Address Space Communication Techniques for Gyrokinetic Fusion Applications on Ultra-Scale Platforms

Robert Preissl
Lawrence Berkeley
National Laboratory
Berkeley, CA, USA 94720
rpreissl@lbl.gov

Nathan Wichmann
Cray Inc.
St. Paul, MN, USA, 55101
wichmann@cray.com

Bill Long
Cray Inc.
St. Paul, MN, USA, 55101
longb@cray.com

John Shalf
Lawrence Berkeley
National Laboratory
Berkeley, CA, USA 94720
jshalf@lbl.gov

Stephane Ethier
Princeton Plasma
Physics Laboratory
Princeton, NJ, USA, 08543
ethier@pppl.gov

Alice Koniges
Lawrence Berkeley
National Laboratory
Berkeley, CA, USA 94720
askoniges@lbl.gov

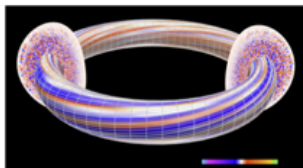


Figure 2: GTS field-line following grid & toroidal domain decomposition. Colors represent isocontours of the quasi-two-dimensional electrostatic potential



Application focus:

- The shift phase of charged particles in a tokamak simulation code



Programming models studied:

- CAF + OpenMP or
- Two-sided MPI + OpenMP



Highlights:

- Experiments on up to 130,560 processors
- 58% speed-up of the CAF implementation over the best multithreaded MPI shifter algorithm on largest scale
- “the complexity required to implement ... MPI-2 one-sided, in addition to several other semantic limitations, is prohibitive.”

CAF at Scale: Weather



BERKELEY LAB

Bringing Science Solutions to the World

Article

A Partitioned Global Address Space
implementation of the European
Centre for Medium Range Weather
Forecasts Integrated Forecasting
System

George Mozdzynski, Mats Hamrud and Nile Wedi

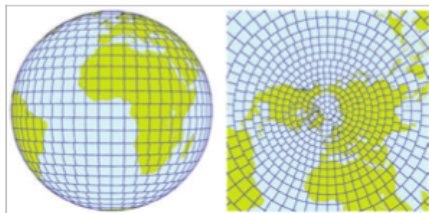


Figure 7. EQ_REGION6 partitioning of grid-point space, showing a partition at the poles and then an increasing number of partitions as we approach the equator.

Mozdzynski, G., Hamrud, M., & Wedi, N. (2015). A partitioned global address space implementation of the European centre for medium range weather forecasts integrated forecasting system. *The International Journal of High Performance Computing Applications*, 29(3), 261-273.



Application:

- European Centre for Medium Range Weather Forecasts (ECMWF) operational weather forecast model



Programming models studied:

- CAF or
- Two-sided MPI



Highlights:

- Simulations on > 60K cores
- performance improvement from switching to CAF peaks at 21% around 40K cores

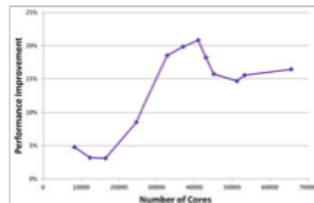
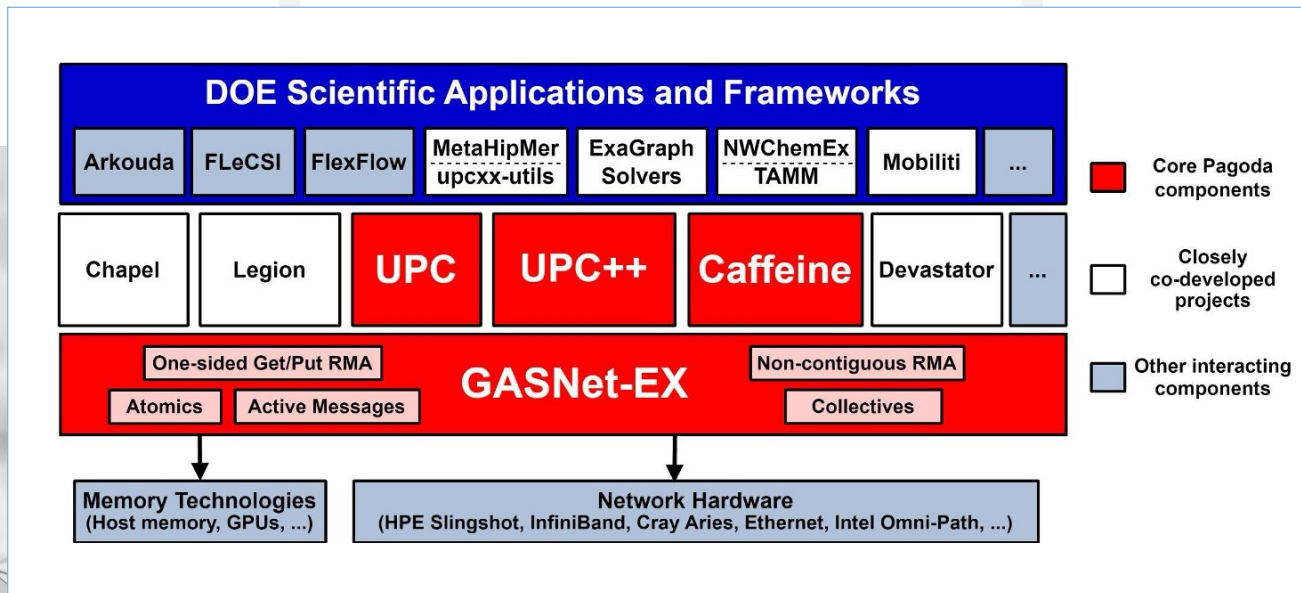


Figure 14. Performance improvement of the TOSM7 10 km model with 107 levels by using Fortran2008 coarray on HECTAR (Cray XE6).

GASNet-EX Ecosystem



Conclusions

Conclusions



PGAS programming models combine programmability with performance.



PGAS models have been applied across many application domains.



Several PGAS models can employ one common communication substrate or different substrates.

Acknowledgements

The Berkeley Lab Fortran Team

Dan Bonachea, Hugh Kadhem, Brad Richardson, Kate Rasmussen

Past and Present Collaborators

Jeremy Bailey, David Torres, Kareem Jabbar Weaver, Jordan Welsman, Yunhao Zhang

Features of Coarrays



BERKELEY LAB

Bringing Science Solutions to the World



Coarray Fortran began as a syntactically small extension to Fortran 95:

- Square-bracketed “cosubscripts” distribute & communicate data



Coarrays integrate with other features:

- Array programming
- OOP: distributed objects



Minimally invasive:

- Drop brackets when not communicating



Communication is explicit:

- Use brackets when communicating

```
81 submodule(subdomain_2D_m) subdomain_2D_s
82 use assertions_m, only : assert
83 implicit none
84
85 real, allocatable :: halo_x(:, :)[:]
86 real dx_, dy_
87 integer, parameter :: west=1, east=2
88 integer my_nx, nx, ny, me, num_subdomains, my_internal_west, my_internal_east
89
90 contains
91
92 module procedure exchange_halo
93   if (me>1) halo_x(east,:)[me-1] = self%s_(1,:)
94   if (me<num_subdomains) halo_x(west,:)[me+1] = self%s_(my_nx,:)
95 end procedure
96
```

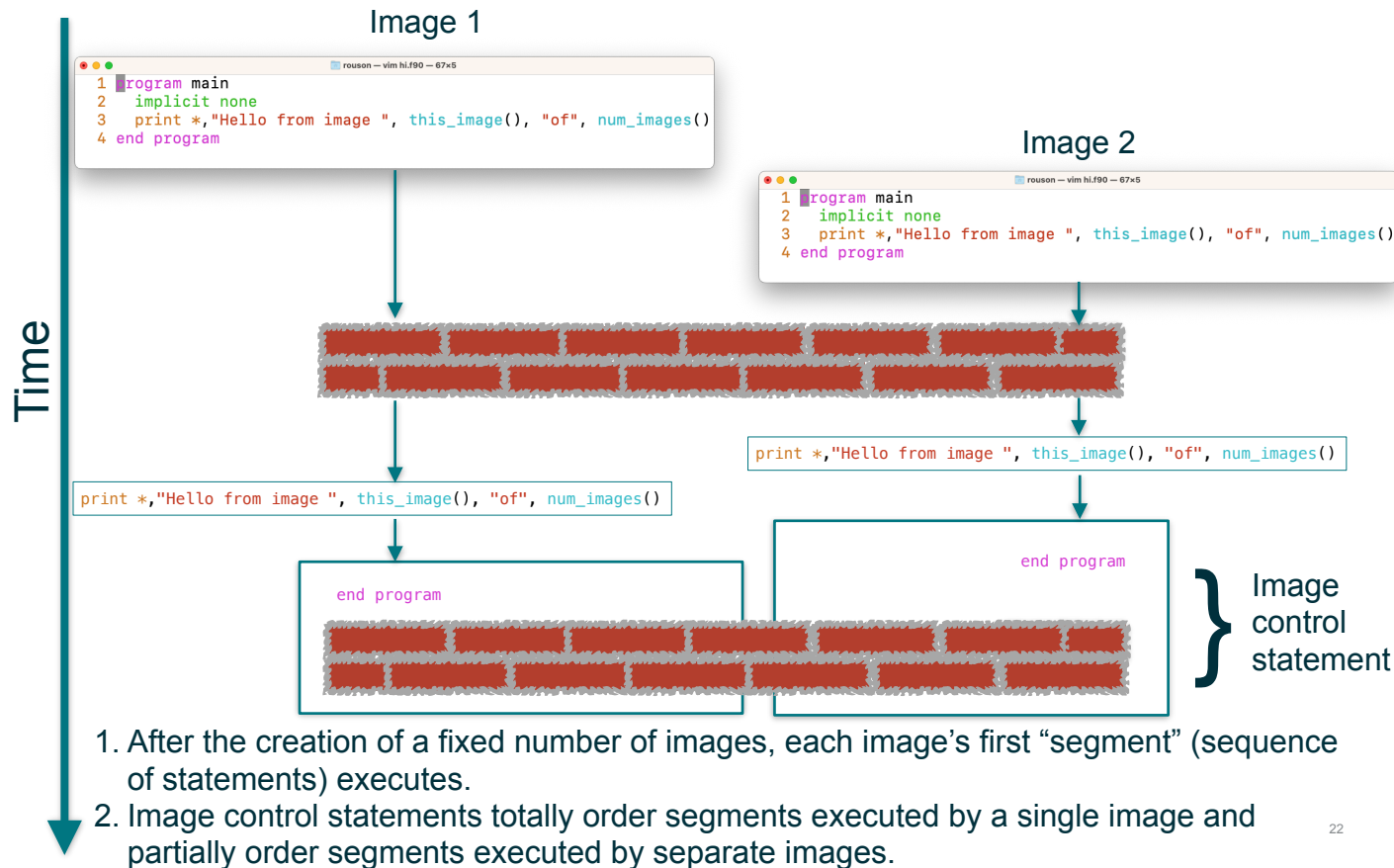
81,1 41%

SPMD Execution Sequence



BERKELEY LAB

Bringing Science Solutions to the World



Explicit Parallelsim in Fortran 2023



BERKELEY LAB

Bringing Science Solutions to the World

Single Program Multiple Data (SPMD) parallel execution

- Synchronized launch of multiple “images” (process/threads/ranks)
- Asynchronous execution except where program explicitly synchronizes
- Error termination or synchronized normal termination

A screenshot of a vim editor window. The title bar shows "rouson - vim hi.f90 - 67x5". The code is as follows:

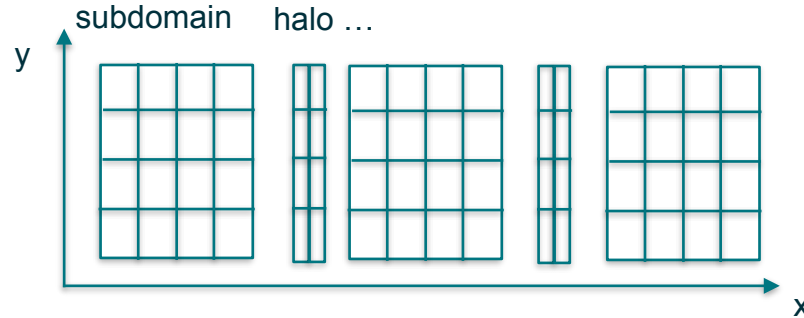
```
1 program main
2   implicit none
3   print *, "Hello from image ", this_image(), "of", num_images()
4 end program
```

Halo Exchange



BERKELEY LAB

Bringing Science Solutions to the World



```
116 real(rkind), allocatable :: halo_x(:, :)[:]
117 integer, parameter :: west=1, east=2

134 me = this_image()
135 num_subdomains = num_images()
137 my_nx = nx/num_subdomains + merge(1, 0, me <= mod(nx, num_subdomains))

232 subroutine exchange_halo(self)
233   class(subdomain_2D_t), intent(in) :: self
234   if (me>1) halo_x(east, :)[me-1] = self%s_(1, :)
235   if (me<num_subdomains) halo_x(west, :)[me+1] = self%s_(my_nx, :)
236 end subroutine
```


CAF at Scale: Climate



BERKELEY LAB

Bringing Science Solutions to the World

Development and performance comparison of MPI and Fortran Coarrays within an atmospheric research model

Extended Abstract

Soren Rasmussen¹, Ethan D Gutmann², Brian Friesen³, Damian Rouson⁴, Salvatore Filippone¹,

Irene Moulitsas¹

¹Cambridge University, UK

²National Center for Atmospheric Research, USA

³Lawrence Berkeley National Laboratory, USA

⁴Sourcery Institute, USA

ABSTRACT

A new application of the Intermediate Complexity Research (ICAR) Model offers an opportunity to compare the scale and performance of the Message Passing Interface (MPI) versus coarray Fortran, two methods of communication across processes. The application requires replicated communication of halo regions, which is performed with either MPI or coarrays. The MPI communication is done using non-blocking isend-recv communication, while the coarray library is implemented using a one-sided MPI or OpenSHMEM communication backend. We observe the development cost in addition to strong and weak scalability analysis to understand the performance costs.

1 INTRODUCTION

1.1 Motivation and Background

In high performance computing MPI has been the de-facto method for memory communication across a system's nodes for many years. MPI 1.0 was released in 1995 and research and development has continued across academia and industry. A method in Fortran 2008, known as coarray Fortran, was introduced to express the communication within the language [6]. This work was based on an extension to Fortran that was introduced by Robert W. Nurnick and John L. Smith [7]. Coarray Fortran, like MPI, is a single-program, multiple-data (SPMD) programming technique. Coarray Fortran's single program is replicated across multiple processes, which can then communicate via MPI or OpenSHMEM.

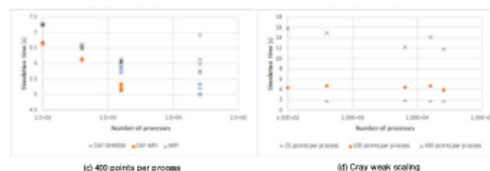


Figure 3: (a-c) Weak scaling results for 25, 100, and 400 points per process (d) weak scaling for Cray.



Application:

- Intermediate Complexity Atmospheric Research (ICAR) model
- Regional impacts of global climate change



Programming models studied:

- CAF over one-sided MPI
- CAF over OpenSHMEM
- Two-sided MPI
- Cray CAF



Highlights:

- “... we used up to 25,600 processes and found that at every data point OpenSHMEM was outperforming MPI.”
- “The coarray Fortran with MPI backend stopped being usable as we went over 2,000 processes... the initialization time started to increase exponentially.”

CAF at Scale: CFD, FFTs, Multigrid



BERKELEY LAB

Bringing Science Solutions to the World

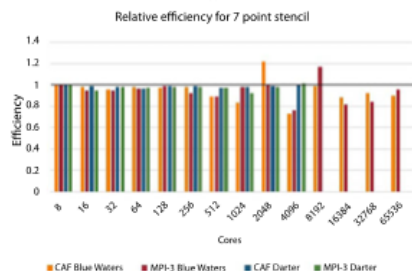


Comparing Coarray Fortran (CAF) with MPI for several structured mesh PDE applications

Sudip Garain^{a,*}, Dinshaw S. Balsara^a, John Reid^b

^aPhysics Department, University of Notre Dame, USA

^bAdvanced Research Laboratory, Oak Ridge, USA



Garain, S., Balsara, D. S., & Reid, J. (2015). Comparing Coarray Fortran (CAF) with MPI for several structured mesh PDE applications. *Journal of Computational Physics*, 297, 237-253.



Applications studied:

- Magnetohydrodynamics (MHD)
- 3D Fast Fourier Transforms (FFTs) used in infinite-order accurate spectral methods
- Multigrid methods with point-wise smoothers requiring fine-grained messaging



Programming models studied:

- CAF or
- One-sided MPI-3



Highlights:

- Simulations on up to 65,536 cores
- “... CAF either draws level with MPI-3 or shows a slight advantage over MPI-3.”
- “CAF and MPI-3 are shown to provide substantial advantages over MPI-2.
- “CAF code is of course much easier to write and maintain...”

Additional Parallel Features



Teams of images can be formed at runtime.



Collective subroutines: `co_{broadcast, sum, max, min, reduce}`



Atomic subroutines:

- `atomic_{define, ref, add, fetch_add, ...}`
- Events: counting semaphores with post/wait/query operations



Failed/stopped image detection, locks, critical sections, ...